

Notes Week 4

a. Dredge-Up Efficiency

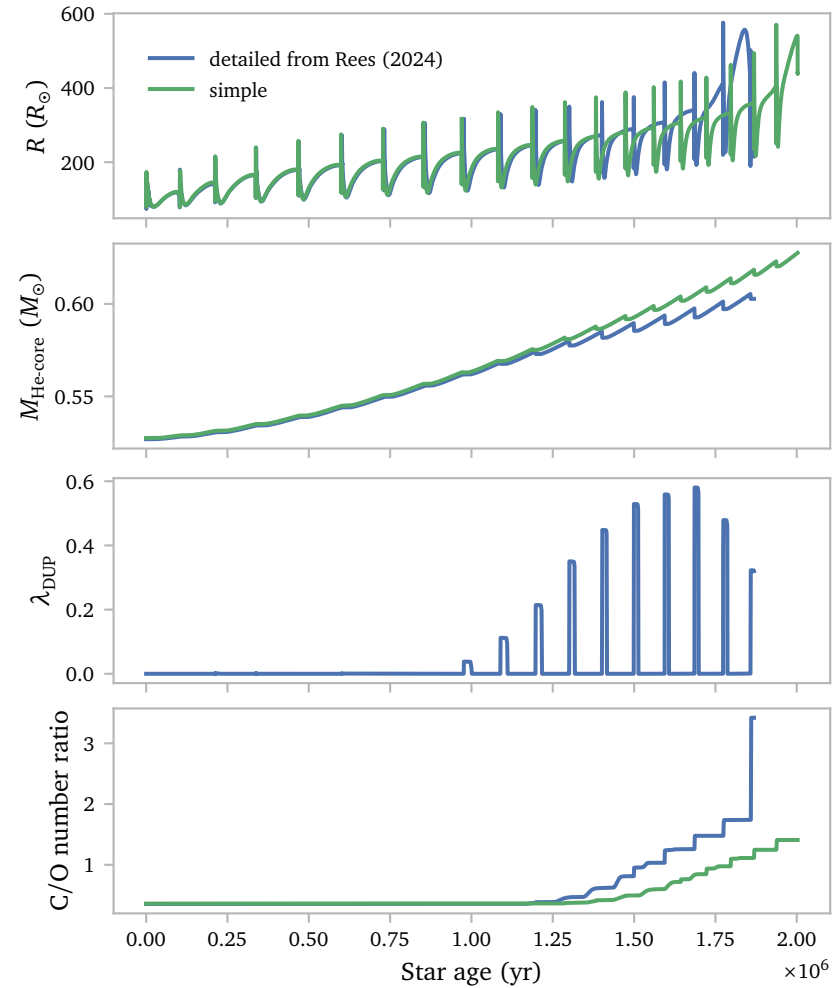
Dredge-up is mixing process that brings materials created during nuclear fusion in the burning regions to the surface due to a (temporary) expansion of the convective envelope to these regions. The first dredge-up occurs after the end of H-core burning. A second dredge-up can occur at the end of He-core burning. The third-dredge up can occur after a helium flash in TPAGB stars (Herwig, 2005).

Although a TPAGB star can undergo numerous thermal pulses, and thus numerous dredge-up episodes, these are all called the third-dredge up. During these episodes, He, C and *s*-process elements are mixed into the envelope. The mixing of C into the envelope changes the C/O ratio, which in turn affects which bands are visible in the spectra of stars. Further more, the C/O ratio affects the opacities¹. The third-dredge up could therefore affect properties of the stellar envelope, where these opacities are most drastically influenced by a changing C/O ratio. Theferofe, it can be interesting to compare dredge-up efficiencies for different MESA models.

The MESA procedure from Rees et al. (2024) is specifically designed to accurately simulate these dredge-up episodes. The efficiency of dredge-up can be expressed by the dredge-up parameter $\lambda = \Delta M_{\text{DUP}} / \Delta M_{\text{H}}$, where ΔM_{DUP} is the mass dredged up after a thermal pulse and ΔM_{H} is the core growth during the preceding interpulse phase.

On the other hand, the mesa model optimized for speed², are not designed with accurately simulating the third dredge-up episodes. It is interesting to compare the models.

Below, I show the radius of the model by Rees et al. (2024) and the simplified model for a $2 M_{\odot}$ star. The plots below show properties related to the third dredge-up episodes.



1 See for example Fig. 10 from Marigo & Aringer (2009)

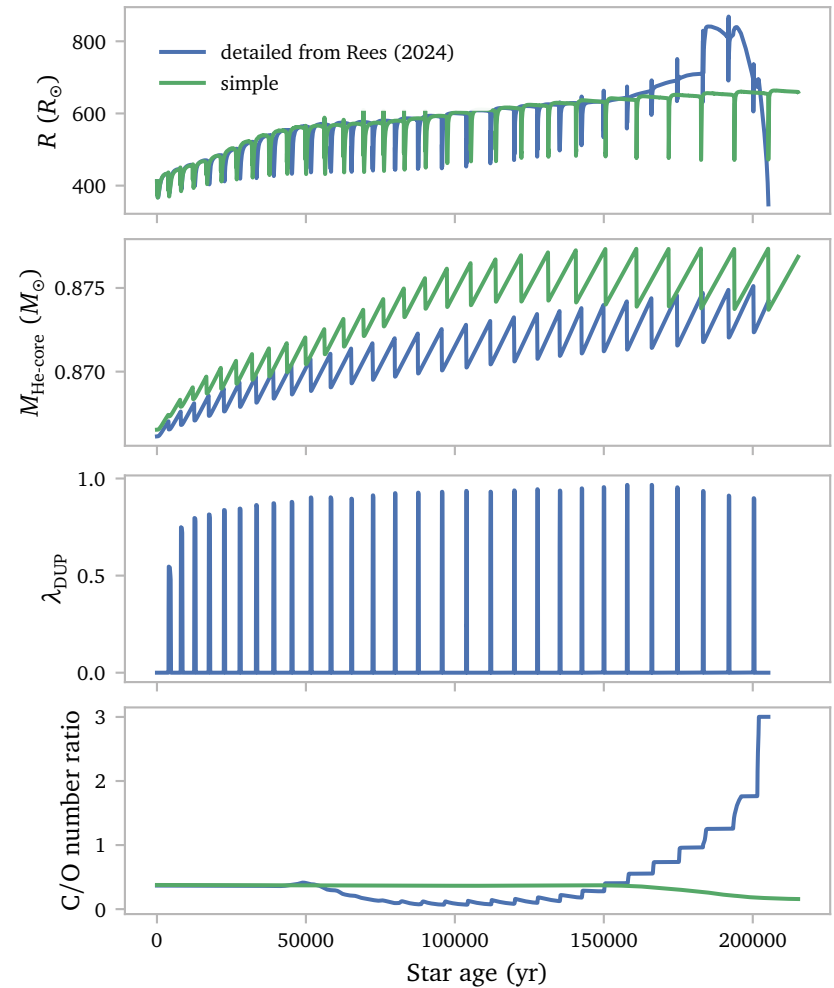
2 By using the default mesa time and mesh resolution parameters, and not explicitly using routines to fully resolve the third dredge-up

The simple model does not contain computed dredge-up efficiencies, but they can be estimated by looking at the change in He-core mass.

The plots seem to imply that the simple model underestimates the dredge-up efficiency. This is in line with the conclusions by Rees et al. (2024), where they state that default MESA settings underestimate the third dredge-up efficiencies by up to 75%.

I also compare models with a hot third dredge-up (HTDU)³. HTDU occurs in stars with masses roughly at and above $5 M_{\odot}$ and results in an initially decreasing C/O ratio, as the hydrogen burning in the convective envelope converts carbon nuclei to nitrogen using the CNO cycle.

This plots below show the same parameters, but now for a $5 M_{\odot}$ star.



3 Near the end of the third dredge-up, the base of the envelope contracts and heats up. In massive AGB stars, the temperature can be sufficiently high for sustained hydrogen burning in this envelope region. This is known as a hot third dredge-up (Rees et al., 2024).

The core mass of the simple model stabilises, meaning $\lambda_{\text{DUP}} \approx 1$. The core mass of the detailed model keeps increasing slightly between thermal pulses, meaning $\lambda_{\text{DUP}} < 1$.

Again, the simple model does not contain computed dredge-up efficiencies.

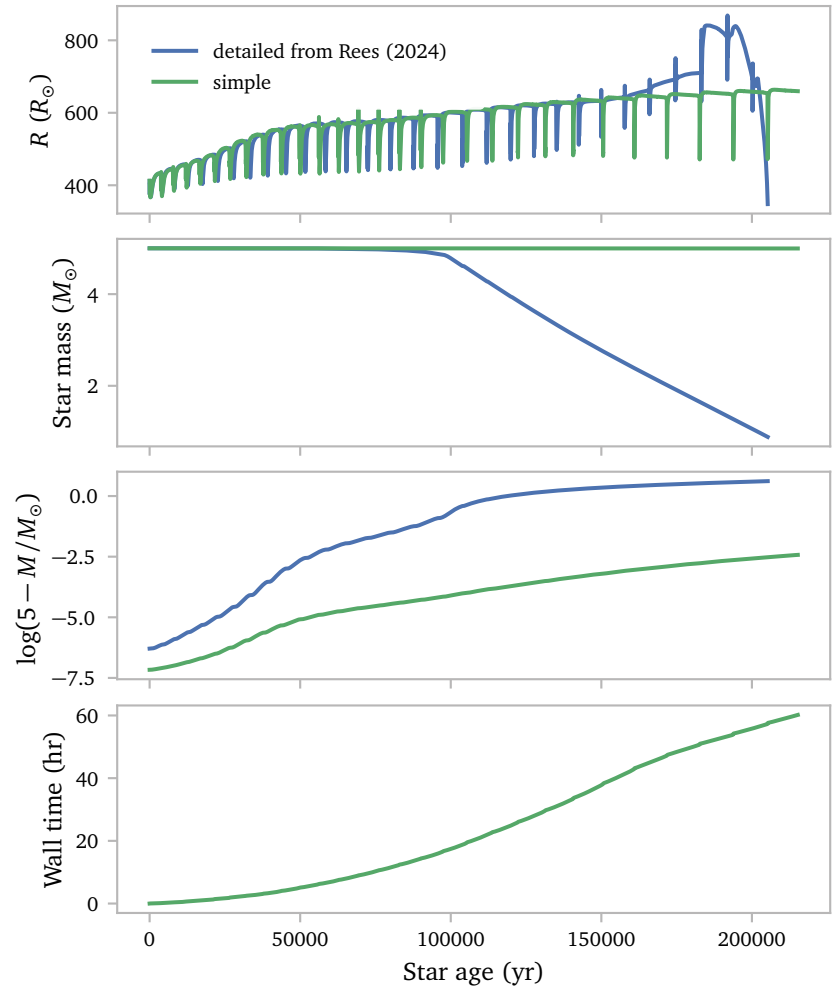
I think the differences mean the simple model does not predict hydrogen burning in the convective envelope during the dredge-up episodes after about 50000 years, while the detailed model does.

This time, the plot of the He-core mass seems to imply, that at least eventually, the simple model overestimates the amount of dredge-up that occurs. This is again in line with the results from Rees et al. (2024), which indicate that HTDU is overestimated by the default parameters of MESA by as much as 55%.

The simple model shows some peaks in the radius during thermal pulses occuring between 50000 and 100000 years after the start of the TPAGB phase. The detailed model does not show these peaks.

The simple model actually took a really long time to evolve as well and did not start losing its envelope even when the detailed star had fully lost its envelope. This might be concerning, especially since the loss of the envelope seems to cause the radius of the star to expand significantly. It is important to note that the simple model used the Vassiliadis & Wood (1993) wind prescription, while the detailed model used the prescription by Trabucchi et al. (2018).

Below, I show the mass evolution and the wall time.



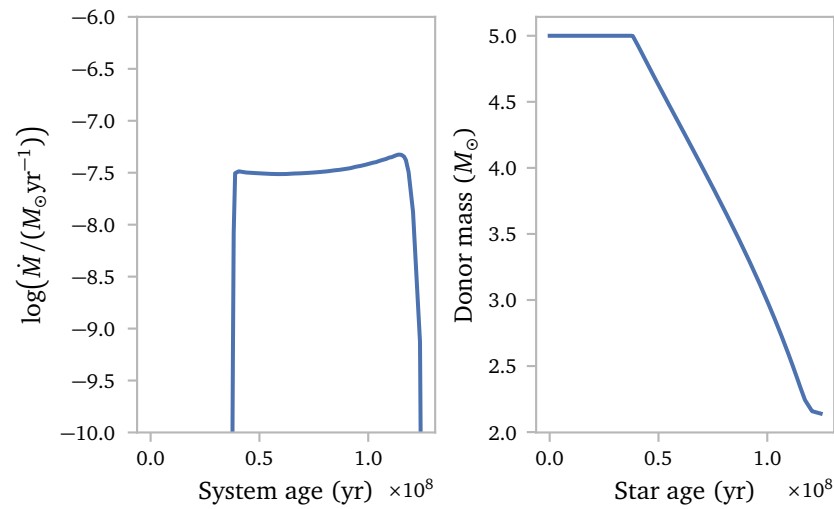
Even though it looks like the simple model is not losing any mass, it actually is, just very little. The plot below shows this.

It seems like it would take a very long time before the envelope will be completely stripped in the simple model. I therefore decided to stop the evolution after 60 hours to free up computing power on my pc.

b. Getting Familiar with the Binary Module

To understand how to effectively use the binary component of MESA, I decided to start with the test suite `star_plus_point_mass`. This test suite contains 4 inlist files, 1 of which is empty and is supposed to represent the point mass star. The main inlist is called `inlist`. It points directly to `inlist_project`. This inlist specifies which inlists should be used for the evolution of the individual stars⁴. This inlist also specifies the masses of the stars. In contrast to the single star case, the masses are not specified in the star inlist files. Then, the inlist specifies an initial separation and some parameters for specifying the exact procedures that are used during mass transfer, including resolution and time controls.

Running the `test_suite` is easy. It produces two history files. One containing the binary evolution history and one containing the single star evolution of the donor. It is then easy to inspect the properties of the binary and single star evolution. For example, below I show the mass loss rate of the donor and the mass of the donor as a function of time⁵.



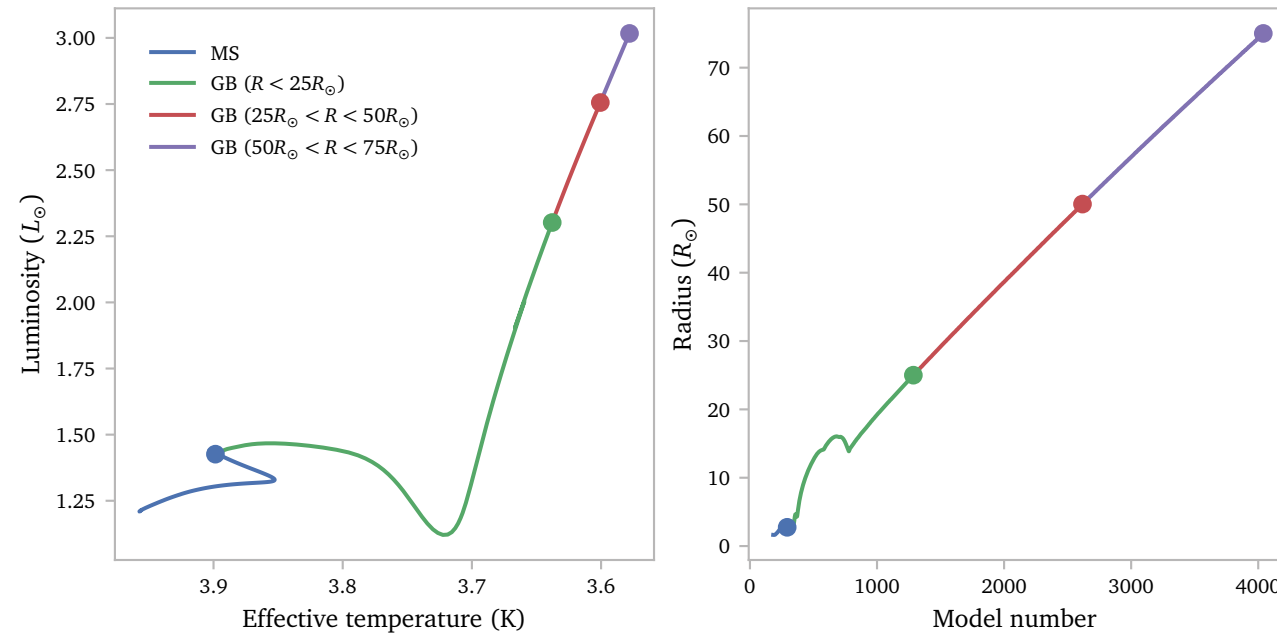
⁴ `inlist1` and `inlist2` respectively. They contain the usual components of inlists of single stars, including all parameters responsible for time and mesh resolution.

⁵ The mass loss rate is obtained from the `binary_history.data` file, while the donor mass is obtained from `history.data`.

The binary module seems to work fine. Now I want to try to start the binary evolution using a model of a star and introduce a point mass later.

To achieve this, let me run a standard 2 solar mass star and evolve it. I use the `hb_2M` test-suite for this purpose. I modified it to evolve over 4 distinct phases. The first phase evolves the star from the pre-MS to the TAMS similar to the original test-suite. The next three phases evolve the star to radii $25M_{\odot}$, $50M_{\odot}$ and $75M_{\odot}$ respectively⁶. After every phase, the simulation saves a `.mod` file, which I hope to use in my binary simulation.

Below, I show the evolution of the simple model from the MS to a radius of $75R_{\odot}$. The dots represent the `.mod` files that are saved at the end of every inlist. I am only interested in the models with the specified radii, which all lie on the GB.



⁶ To achieve this, I copied the `inlist_to_ZACHeB` inlist 3 times and simply changed the stopping condition in these files to `log_Rsurf_upper_limit = <value>`. I then modified the `rn` file to run the new inlists in the correct order.

I then save the models and the inlists in a new directory containing the binary component of MESA and a new inlist containing the binary settings, largely based on the binary inlist by [Temmink et al. \(2023\)](#). It is important to specify a correct initial separation, such that the star starts its evolution very close to filling its Roche lobe. As a starting point, I use that the initial separation should be such that $R/R_L \approx 0.9$. To find the orbital separation that produces this ratio, I use the Roche lobe fitting formula from [Eggleton \(1983\)](#),

$$\frac{R_L}{a} \approx \frac{0.49q^{-2/3}}{0.6q^{-2/3} + \ln(1 + q^{-1/3})}.$$

For simplicity, and the expected stability according to [Temmink et al. \(2023\)](#), I use a mass ratio $q \equiv M_a/M_d = 1$. The Roche lobe fitting formula reduces to

$$R_L \approx \frac{0.49a}{0.6 + \ln(2)}.$$

The final separation becomes⁷

$$a \approx 2.9323R.$$

Starting with our $25R_{\odot}$ star, this would equate to an initial separation of

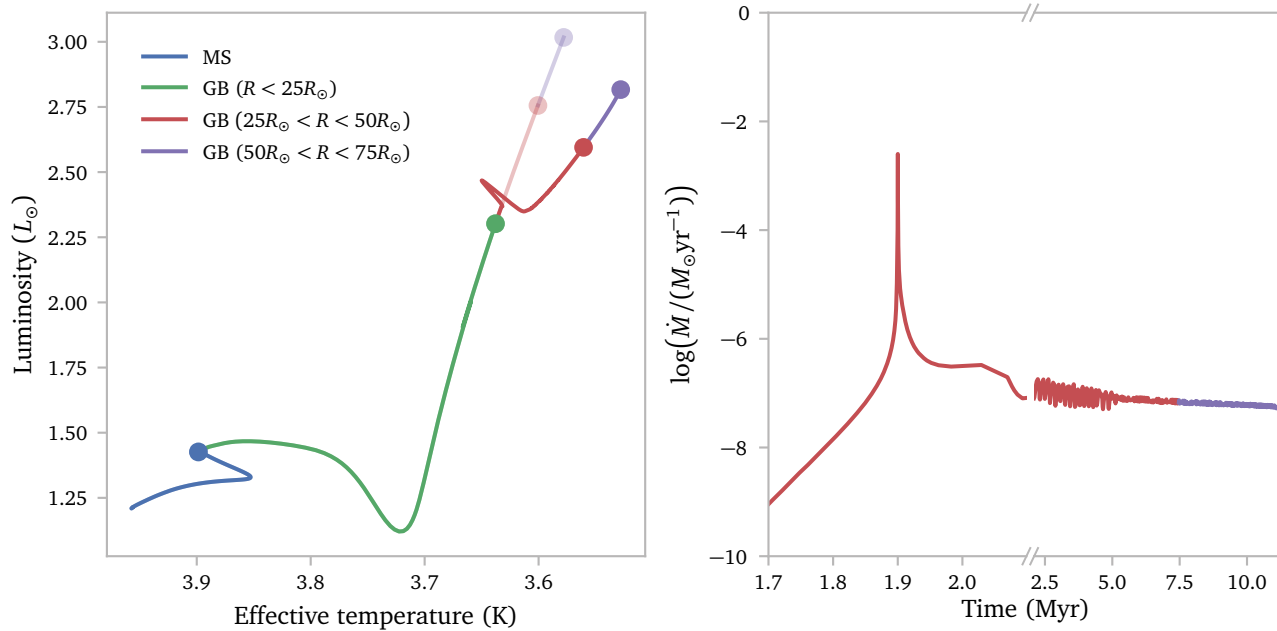
$$a \approx 73.308R_{\odot}.$$

⁷ Using the equation for the Roche lobe and the radius ratio criterion:

$$\frac{R}{R_L} \approx \frac{0.6R + R \ln 2}{0.49a} = 0.9$$

$$0.6R + R \ln 2 = 0.441a.$$

Below, I plot the evolution of the star in the HR-diagram as well as its mass loss rate over time.



The right subplot looks very similar to the results of [Temmink et al. \(2023\)](#) Figure 5. We recognise the peak in the mass transfer rate that is typical for donor stars with convective envelopes⁸.

I notice some oscillations in the mass transfer rate after the initial peak. I am not sure if this is caused by the crude default MESA single star GB evolution, an inaccuracy in binary mass transfer prescription, or an actual physical process.

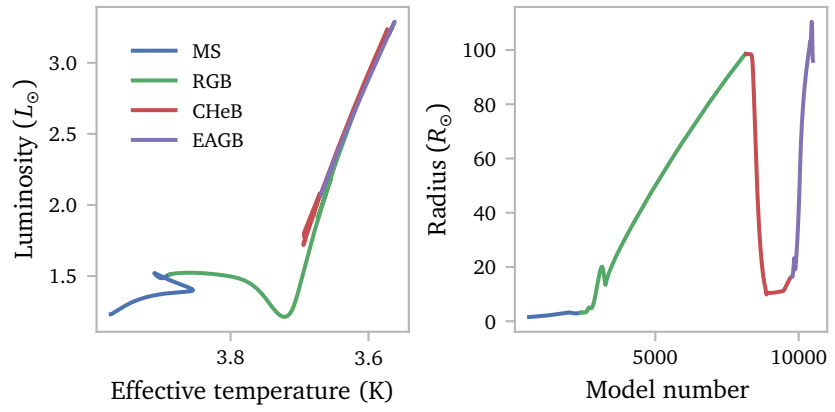
Next, I want to simulate mass transfer on the EAGB branch using the detailed model by [Rees et al. \(2024\)](#) for the single star evolution.

⁸ I am not sure what actually causes the drop in mass transfer rate again

c. Selecting a Model for Mass Transfer

In order to prepare for simulating mass transfer during the TPAGB phase, we want to try to simulate mass transfer for a phase where mass transfer stability is better understood. The work by [Temmink et al. \(2023\)](#) has shown how MESA can be used to simulate mass transfer (stability) in binary stars with giant type donors. We want to apply the same techniques.

For this, we need a starting model to start the binary evolution with. Below, I show the evolution of the $2M_{\odot}$ star by [Rees et al. \(2024\)](#).



Notice how only a very tiny part of the evolution on the EAGB has a larger radius than the maximum radius on the RGB.

The radial evolution of a $2M_{\odot}$ mass star makes it very unlikely that this star will interact during the EAGB phase. The work by [Temmink et al. \(2023\)](#) shows that a slightly heavier star will be more likely to interact on the EAGB phase. As a first test, I therefore decided to simulate a $3M_{\odot}$ star and a $2.5M_{\odot}$ star using the prescription by [Rees et al. \(2024\)](#).

I modify the `run_star_extras.f90` in order to save models every time the radius increases.

Below, I note the steps I took:

but i show it here for clarity.

```
! src/run_star_extras.f90
module run_star_extras

use star_lib
use star_def
use const_def
use math_lib

implicit none

integer :: evol_phase

include 'agb/agb_params.inc'
integer, parameter :: x_last_saved_R = 6

contains
...
```

This line is actually included in `agb/agb_params.inc`

MESA is now aware that it should save a sixth extra real value inside the star object. The other 5 reals are values that [Rees et al. \(2024\)](#) define and use. I do not initialize this real in subroutine `extras_startup(id, restart, ierr)` since the default value of 0 is sufficient.

I then modify the extra code that runs after every completed step:

```
! src/agb/agb_run_star_extras.inc
integer function extras_finish_step(id)
  integer, intent(in) :: id
  integer :: ierr
  type (star_info), pointer :: s
  logical :: do_termination
  character(len=50) :: do_termination_code_str

  ! declare variables for saving models
  character (len=200) :: fname
  real(dp) :: R_now, R_last
  real(dp) :: h1c

  ierr = 0
  call star_ptr(id, s, ierr)
  if (ierr /= 0) return
  extras_finish_step = keep_going

  ! write a new model every time the radius has increased
  ! by 10% with respect to the previous saved model.
  R_now = s% photosphere_R
  R_last = s% xtra(x_last_saved_R)

  h1c = s% center_h1

  if (.not. (h1c == h1c)) then
    h1c = 1d0
  end if

  if ((R_now > 1.1d0 * R_last) .and. (h1c < 0.68d0)) then
    write(fname, fmt="(a12,f0.3,a8)") 'LOGS/models/', R_now, 'Rsun.mod'

    ! write model, profile and history
    call star_write_model(id, fname, ierr)
    s% need_to_update_history_now = .true.
    s% need_to_save_profiles_now = .true.

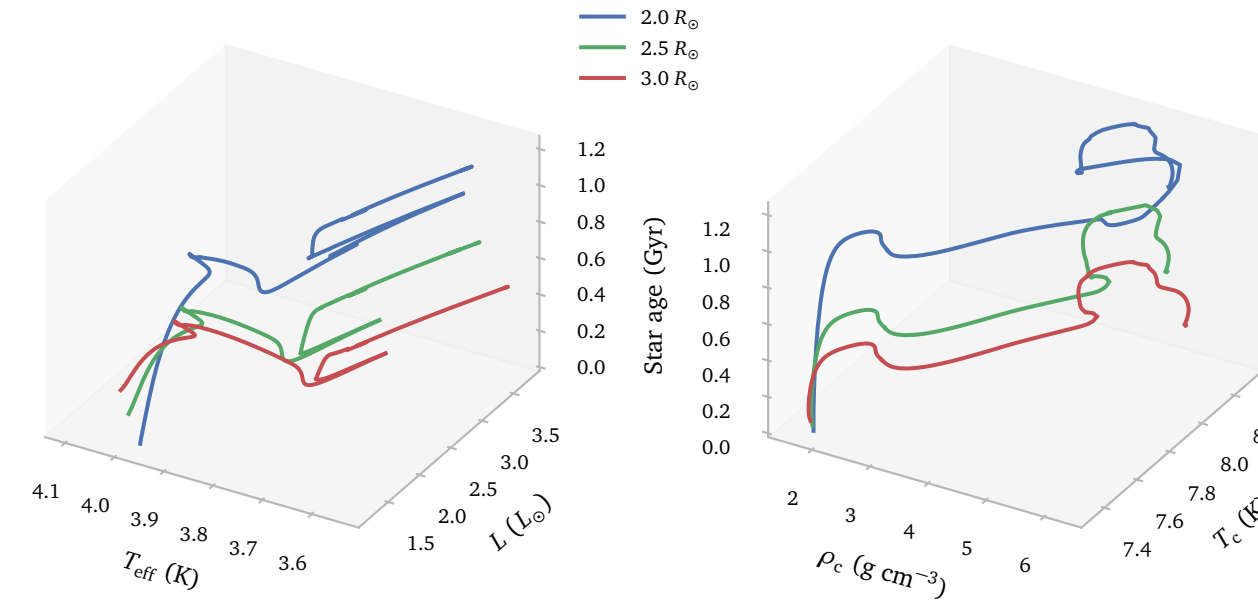
    if (ierr /= 0) then
      write(*,*) 'error writing model'
      ierr = 0
    end if

    s% xtra(x_last_saved_R) = R_now
  end if

  ...

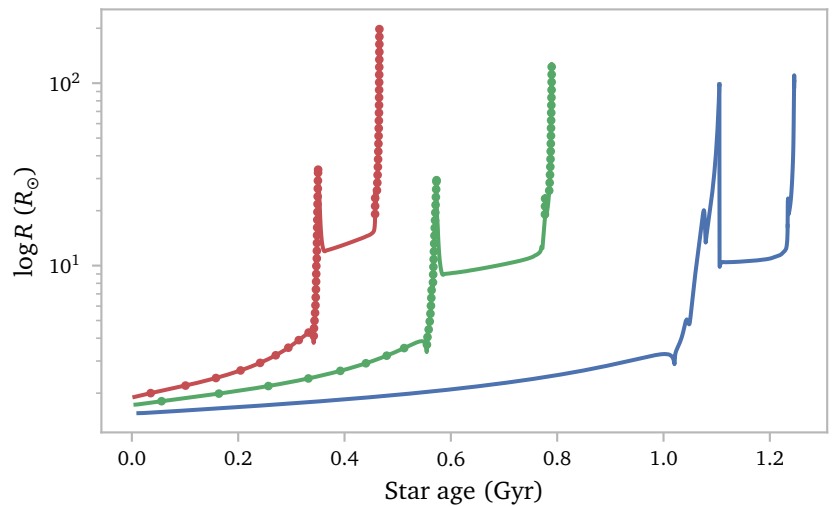
end function extras_finish_step
```

This code ensures models are saved every time the radius of the star exceeds the radius of the last saved model. Below, I show the evolution of the two new models compared to the $2M_{\odot}$ model.



The figure shows that the $2M_{\odot}$ star obtains a degenerate core and undergoes a He-flash, which causes a spike in luminosity (and radius) and a steep rise in central density and temperature. The heavier stars do not experience this flash, and thus maintain a smaller radius on the red giant branch and the following CHeB phase. This increases the probability of mass transfer occurring on the EAGB phase ⁹.

Below, I show a plot of the radius evolution of the three stars, with markers where models have been saved for the new stars.



First things first, it works! MESA saved models every time the radius increased and I now have models that I can use to simulate mass transfer on the EAGB phase. The $3M_{\odot}$ model looks particularly well suited for this. However, I notice two things.

1. Saving a model when the radius increases by just 10% results in too many models. For now, I will just increase the percentage to 25%¹⁰.
2. The custom star variable `last_saved_R` gets set to 0 after every inlist change. This results in saved models that *do not* have larger radii than the maximum radius during a previous phase.

⁹ This is a phase of helium shell burning before the onset of thermal pulses.

¹⁰ Saving every $\Delta \log R_d = 0.2$, like [Temmink et al. \(2023\)](#) did, is probably sufficient for a broader study. Saving every 25% increase will result in roughly double the number of models.

d. Fixing “Problems”

The first “problem” can easily be fixed by simply changing the condition for saving models to

```
if ((R_now > 1.25d0 * R_last) .and. (h1c < 0.68d0)) then.
```

In order to solve the second “problem”, we need to modify the way MESA initializes `xtra(x_last_saved_R)` in `extras_startup`.

We want MESA to always use the last saved R, even when starting a new run with a different inlist¹¹. I therefore write a simple file every time the last saved radius gets updated:

```
! agb/agb_run_star_extras.inc
...
call star_write_model(id, fname, ierr)
s% need_to_update_history_now = .true.
s% need_to_save_profiles_now = .true.
open(1, file = "last_saved_R.dat", status="replace")
write(1,*) R_now
close(1)
...
```

Then, in the `extras_startup`:

```
! agb/agb_run_star_extras.inc
subroutine extras_startup(id, restart, ierr)
  integer, intent(in) :: id
  logical, intent(in) :: restart
  integer, intent(out) :: ierr
  type (star_info), pointer :: s

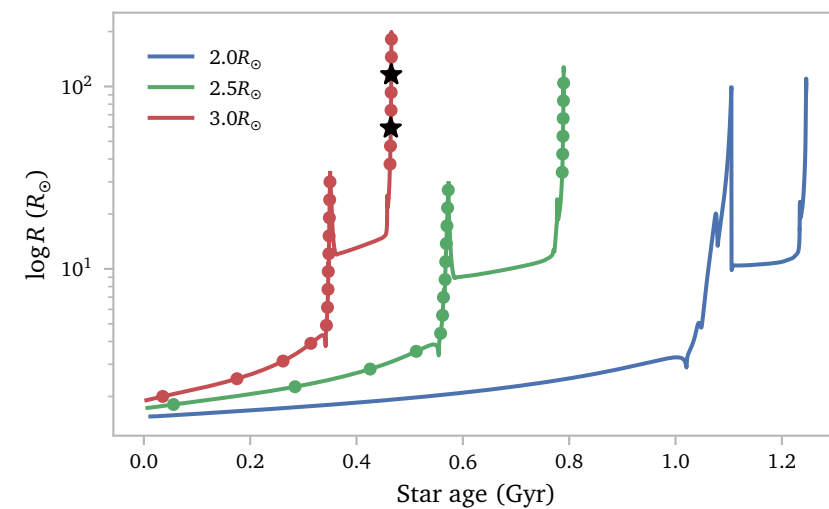
  integer :: ios
  real(dp) :: stored_R

  ierr = 0
  call star_ptr(id, s, ierr)
  if (ierr /= 0) return

  open(1, file='last_saved_R.dat', status='old',
       ↪ action='read', iostat=ios)
  if (ios == 0) then
    read(1,*,iostat=ios) stored_R
    if (ios == 0) then
      s%xtra(x_last_saved_R) = stored_R
      write(*,*) 'loaded last_R = ', stored_R
    end if
    close(1)
  else
    write(*,*) 'no last_R.dat found, initializing with
    ↪ 0'
    s%xtra(x_last_saved_R) = 0
  end if
  ...
end subroutine extras_startup
```

This code ensures that the file gets read, *or*, the value gets initialized to 0 in the case that the file does not exist.

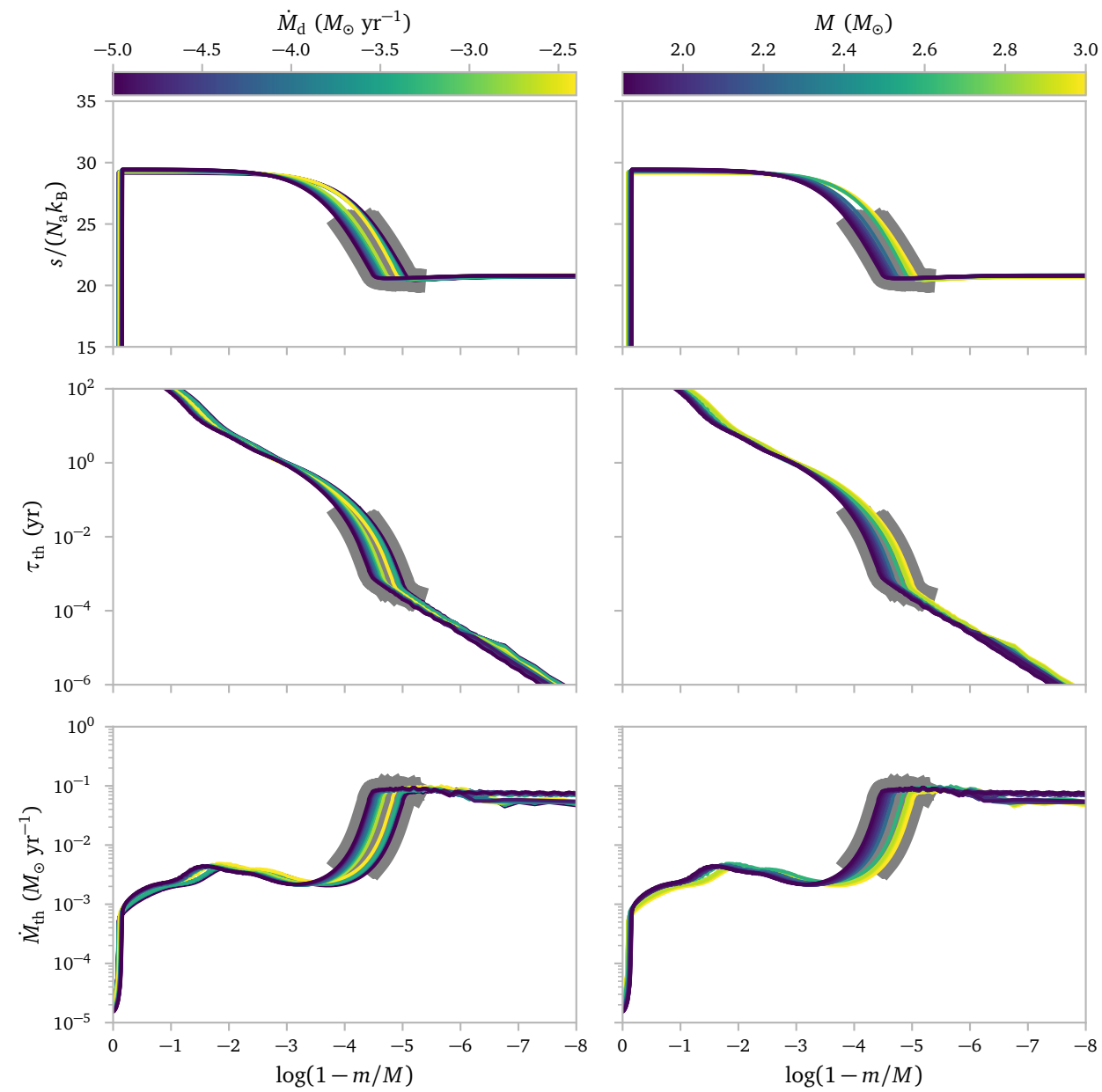
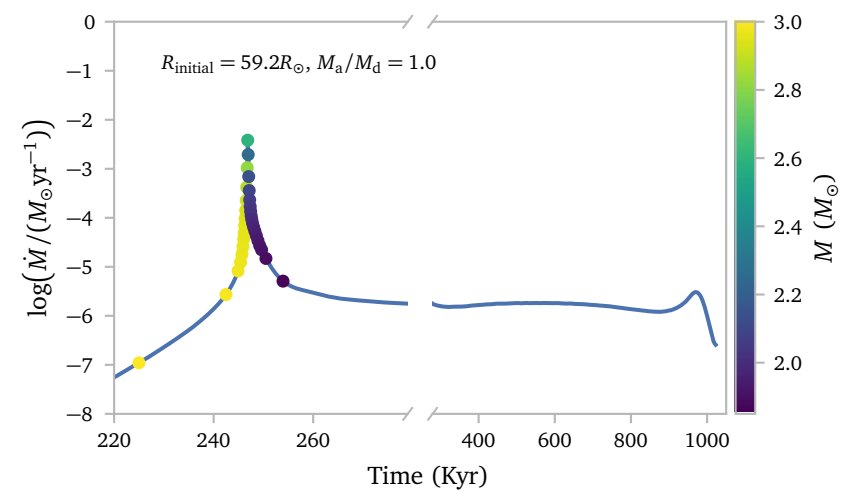
Below, I show the updated results.



I can now freely choose a model on the EAGB branch and use it as a starting model for the binary evolution.

¹¹ For example, when the MS evolution is complete, the model will terminate, the inlist will be swapped and a new run is started.

e. Results!

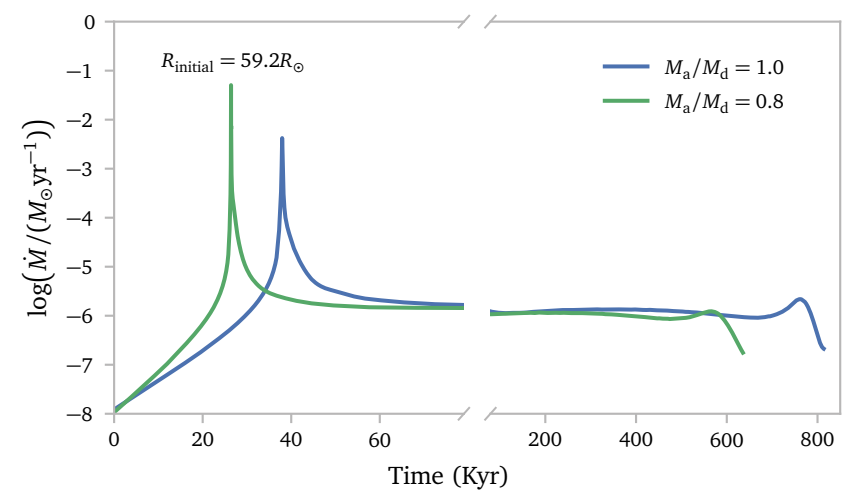


For varying q , we need the full expression from [Eggleton \(1983\)](#).

$$\frac{R}{R_L} = \frac{0.6 \cdot q^{-2/3} R + R \cdot \ln(1 + q^{-1/3})}{0.49 \cdot q^{-2/3} a} = 0.9$$

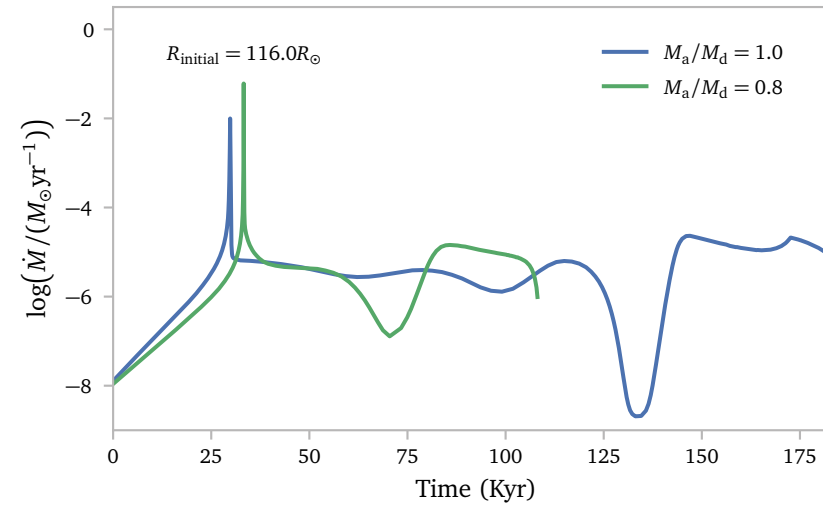
$$a = \frac{0.6 \cdot q^{-2/3} R + R \cdot \ln(1 + q^{-1/3})}{0.411 \cdot q^{-2/3}}.$$

For the $59.194R_\odot$ model and a mass ratio of $q = M_a/M_d = 0.8$, this yield an initial separation of $a = 177.147R_\odot$.



Nice!

f.



References

- Eggleton, P. P. (1983) *Approximations to the radii of Roche lobes.* [ApJ268, 368](#).
- Herwig, F. (2005) *Evolution of Asymptotic Giant Branch Stars.* [ARA&A43, 435](#).
- Marigo, P. & Aringer, B. (2009) *Low-temperature gas opacity. ÆSOPUS: a versatile and quick computational tool.* [A&A508, 1539](#).
- Rees, N. R., Izzard, R. G., & Karakas, A. I. (2024) *Thermal pulses with MESA: resolving the third dredge-up.* [MNRAS527, 9643](#).
- Temmink, K. D., Pols, O. R., Justham, S., Istrate, A. G., & Toonen, S. (2023) *Coping with loss. Stability of mass transfer from post-main-sequence donor stars.* [A&A669, A45](#).
- Trabucchi, M., Wood, P. R., Montalbán, J., et al. (2018) *Long-Period Variables in the Large Magellanic Cloud.* [514, 111](#).
- Vassiliadis, E. & Wood, P. R. (1993) *Evolution of Low- and Intermediate-Mass Stars to the End of the Asymptotic Giant Branch with Mass Loss.* [ApJ413, 641](#).